Course Outline: Databases

Title: Databases: A Map of the Landscape **Skill:** Skill 30 — Consultar tablas y gestionar filas en bases de datos **Learnpack:** + Bases de datos **File:** s30-w11d31-bases-de-datos **Prerequisite:** Unit testing en Typescript (s30-w10d30) **Target Audience:** Beginner developers who have used TinyDB for data persistence and are ready to understand the full database ecosystem **Total Lessons:** 9 **Format:** Conceptual (0% hands-on)

Course Description

Students have already been persisting data with TinyDB — a document database — without knowing how it fits into the broader landscape of database technologies. This course provides that map. Students will understand what databases are designed to solve, how different types of databases model data differently, and when to reach for each type. The course covers relational databases (the SQL family), document databases, key-value stores, and a brief look at graph and columnar databases — giving students the vocabulary and mental model they need before working with real SQL databases in upcoming courses.

Course Philosophy

This course emphasizes:

- Building on what students already know (TinyDB) rather than starting from scratch
- Understanding tradeoffs over memorizing technology names
- Developing the habit of asking "what problem does this tool solve?" before choosing any technology

---

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Relational Databases	2
02 - Non-Relational Databases	3
03 - Putting It All Together	2
04 - Outro	1
Total	**9**

Section 00: Welcome

00.0 The Database Landscape 🕮

Learning Objectives:

- Recognize that TinyDB, which students have already used, is one type of database among many
- Understand what problem all databases share in common: structured, persistent, queryable storage
- Preview the categories of databases covered in this course

Content Outline:

- The problem all databases solve: storing data in a way that can be reliably retrieved, queried, and updated — outlasting any single program run
- How this differs from files: structure, querying, concurrent access, integrity
- A brief, engaging taxonomy preview: relational, document, key-value, and specialized types
- Why so many types exist: different data shapes, different access patterns, different scale requirements
- What students will be able to say by the end: "I know what kind of database fits what kind of problem"

Transitions: This lesson sets the frame. Students leave knowing that the full picture is richer than TinyDB — and curious about what they've been missing.

Key Principles:

- All databases exist to solve the same core problem: reliable, structured, persistent storage
- Different database types are different answers to the question "what shape is your data, and how will you access it?"

Examples:

- A contact list stored in a CSV file vs. stored in a database: what breaks at scale, what becomes trivial
- TinyDB as a concrete anchor: students already know one database; the course will show them where it lives on the map

---

Section 01: Relational Databases

01.0 The Relational Model 🕮

Learning Objectives:

- Explain what makes a database "relational"
- Describe how data is structured in tables, rows, and columns
- Understand the role of primary keys and the concept of relationships between tables (at a conceptual level — not SQL syntax)

Content Outline:

- The core idea: data organized into tables (relations), each row is a record, each column is an attribute
- Primary keys: every row has a unique identifier
- Relationships: how tables reference each other (conceptual — foreign keys as an idea, not syntax)
- Why this model works: consistency, querying power, mature tooling, decades of battle-testing
- The tradeoff: rigid schema — you define the shape of your data upfront; changing it later requires care

Transitions: Now that students understand what "relational" means conceptually, the next lesson introduces the actual systems in this family — the names they'll encounter in the real world.

Key Principles:

- A relational database enforces structure: every record of the same type has the same shape
- The relational model's power comes from its constraint: you can query across related tables reliably because the relationships are explicit and enforced
- Schema rigidity is not a flaw — it's a feature that ensures data integrity

Examples:

- A `users` table and an `orders` table: each order belongs to a user via a reference — no data is duplicated, the relationship is stored once
- Comparing a spreadsheet (flat, no relationships) to a relational database (tables that reference each other)

01.1 The SQL Family 🕮

Learning Objectives:

- Identify the major relational database systems: PostgreSQL, MySQL/MariaDB, SQL Server
- Describe the key characteristics and typical use cases of each
- Understand that SQL is the language shared across these systems, even though each system has its own dialect

Content Outline:

- SQL as the shared language of relational databases — one skill transfers across systems
- **PostgreSQL:** open source, feature-rich, widely used in production web applications, excellent standards compliance, strong community. The industry default for new projects.
- **MySQL / MariaDB:** MySQL is one of the most deployed databases in the world (historically dominant in web hosting); MariaDB is its open-source fork. Widely used with PHP/WordPress ecosystems. MariaDB emerged as a community-driven alternative when Oracle acquired MySQL.
- **SQL Server:** Microsoft's enterprise offering. Common in corporate and Windows-heavy environments. Often found in large enterprises and .NET stacks.
- The practical reality: in most web development contexts, PostgreSQL is the default choice today — but encountering the others is inevitable

Transitions: Students now know the relational family. Next, they'll return to familiar territory: document databases, where TinyDB lives — and discover two more members of that family.

Key Principles:

- Choosing between relational systems is often about ecosystem, hosting environment, and team familiarity — the relational model itself is shared
- PostgreSQL is the safe default for new projects without specific constraints
- SQL knowledge transfers: learn it once, adapt to any system's dialect

Examples:

- A startup building a new web app: likely reaches for PostgreSQL + a cloud provider (Heroku, Railway, Supabase)
- A company running a legacy WordPress site: likely running MySQL under the hood
- A large bank or insurance company: may be running SQL Server integrated with other Microsoft tooling

---

Section 02: Non-Relational Databases

02.0 Document Databases 🕮

Learning Objectives:

- Explain what a document database is and how it differs from a relational database
- Recognize TinyDB and MongoDB as document databases
- Understand when document databases are a good fit and when they aren't
- Clarify SQLite's position: relational model, but file-based and embedded

Content Outline:

- The document model: data stored as self-contained documents (JSON-like objects), not rows in a table
- No fixed schema: each document can have a different shape — flexible for evolving data
- TinyDB revisited: students have already used this. TinyDB stores data as JSON documents in a local file. That *is* a document database — a minimal one.
- **MongoDB:** the most widely used document database in production. Documents are stored as BSON (binary JSON). Designed for scale, with rich querying capabilities.
- **SQLite:** a clarification worth making — SQLite is actually relational (it uses SQL), but it's embedded and file-based, which makes it feel "lightweight" like TinyDB. Often grouped with simple storage options. Important distinction: it's a SQL database, not a document store.
- When document databases shine: content management, user profiles, product catalogs — data with variable shapes or nested structures
- When they fall short: highly interconnected data with complex relationships (relational is better there)

Transitions: Document databases store rich, structured objects. The next type takes a completely different approach: ruthless simplicity — a key maps to a value, nothing more.

Key Principles:

- Document databases trade schema enforcement for flexibility — great for data that doesn't fit neatly into rows and columns
- TinyDB is a document database: students have already been using this paradigm
- SQLite is relational despite feeling "lightweight" — don't confuse file-based with document-based

Examples:

- A user profile with optional fields (some users have a phone number, some don't, some have multiple addresses): fits naturally in a document, awkwardly in a rigid table
- MongoDB storing a blog post as one document: title, content, tags array, author object, comments array — all nested, no joins needed
- TinyDB storing contacts as a list of JSON objects: exactly the document model, just at a tiny scale

---

02.1 Key-Value Stores 🕮

Learning Objectives:

- Explain the key-value data model: a key maps to a value, that's it
- Describe Redis and its primary use cases
- Understand why extreme simplicity enables extreme speed

Content Outline:

- The key-value model: the simplest possible data structure — you store a value under a key, you retrieve it by that key. No relationships, no schema, no querying by content.
- Why this exists: sometimes you don't need to query — you just need fast lookup. Cache a result. Store a session. Track a counter.
- **Redis:** the dominant key-value store. Runs entirely in memory (with optional persistence to disk), making it extremely fast. Supports more than plain strings: lists, sets, sorted sets, hashes — but always accessed by key.
- Common Redis use cases:
  - **Caching:** store expensive query results so you don't recompute them on every request
  - **Sessions:** store user session data server-side (key = session ID, value = session data)
  - **Rate limiting:** track how many requests a user has made in a time window
  - **Pub/Sub messaging:** real-time event broadcasting between services
- The tradeoff: you can't ask "find all values where the value contains X" — if you don't know the key, you can't query

Transitions: Relational, document, key-value — three very different answers to the same problem. There are two more specialized types worth knowing by name, even if they're rarely the first tool you reach for.

Key Principles:

- The key-value model's power is its constraint: by giving up query flexibility, it gains speed

- Redis lives in memory — that's why it's fast, and why it's used for ephemeral data (caches, sessions) rather than primary storage
- "I need to look this up by a known key, fast" → key-value store. "I need to search by content" → something else.

Examples:

- A web app caches the result of a slow database query in Redis with a 5-minute expiration: the next 1000 requests skip the database entirely
- A session ID stored in Redis: `session:abc123` → `{userId: 42, role: "admin", expiresAt: ...}`
- A rate limiter: `ratelimit:user:42` → `17` (number of requests in the last minute)

02.2 Other Database Types 📖

Learning Objectives:

- Recognize graph and columnar databases as specialized tools for specific problems
- Describe the problem each type was designed to solve
- Understand that specialized database types exist because no single model fits every data shape

Content Outline:

- The pattern so far: every database type is a tradeoff optimized for a specific data shape and access pattern. Two more types complete the picture.
- **Graph databases (e.g., Neo4j):** data is stored as nodes and edges — entities and the relationships between them. When the *relationships* are the data (not just a means to link records), graphs shine.
    - Use case: social networks ("who are the friends of my friends?"), fraud detection (following chains of transactions), recommendation engines
    - Why relational falls short here: deeply nested JOIN chains become slow and complex; graphs traverse relationships natively
- **Columnar databases (e.g., Apache Cassandra, Amazon Redshift):** data is stored by column, not by row. Optimized for reading large amounts of data across many records for a small number of columns — analytical workloads.
    - Use case: data warehousing, analytics, time-series data at scale (IoT sensors, logs)
    - Why relational falls short here: scanning millions of rows to aggregate one column is slow when you have to read the full row each time; columnar skips unneeded columns
- The takeaway: these are specialized tools. Most web developers won't reach for them on a typical project — but knowing they exist helps you recognize when a problem might need them.

Transitions: Students now have a complete map of the database landscape. The final content lesson asks the practical question: given all of this, how do you actually choose?

Key Principles:

- Graph databases make *relationships* first-class citizens — not an afterthought stored as foreign keys

- Columnar databases are built for analytics, not transactions — reading wide datasets fast, not writing individual records fast
- Knowing a tool exists is different from needing to use it — but the knowledge prevents you from reaching for the wrong hammer

Examples:

- LinkedIn's "people you may know" feature: a graph traversal problem — find nodes (people) connected within 2 degrees
- A business intelligence dashboard aggregating 3 years of sales data by product category: columnar database reads only the `category` and `amount` columns across millions of rows, skipping everything else
- A startup building a to-do app: absolutely does not need a graph database or a columnar store — a relational database is the right tool

---

Section 03: Putting It All Together

03.0 Choosing the Right Database 🕮

Learning Objectives:

- Apply a decision framework to choose an appropriate database type for a given scenario
- Identify the key questions to ask when evaluating database options
- Recognize that the right choice depends on data shape, access patterns, scale, and ecosystem constraints

Content Outline:

- The decision framework — three questions to ask:
    1. **What shape is your data?** Fixed schema with clear relationships → relational. Variable, nested, self-contained objects → document. Simple key-based lookup → key-value. Deeply interconnected relationships as the focus → graph. Analytical queries across huge datasets → columnar.
    2. **How will you access it?** Need to query by any field, filter, sort, join → relational or document. Need blazing-fast lookup by a known key → key-value. Need to traverse relationships → graph.
    3. **What's your ecosystem?** What does your team know? What does your infrastructure support? What does your framework integrate with best?
- The "boring default" principle: for most web applications, a relational database (PostgreSQL) is the right starting point. Reach for something else only when you have a concrete reason.
- Common mistakes:
    - Choosing a trendy database over a proven one without a concrete reason
    - Using a key-value store as a primary database (it's not designed for that)
    - Picking a graph database for data that is actually just relational with a few joins
- Polyglot persistence: large systems often use multiple database types together — PostgreSQL for core data, Redis for caching, MongoDB for a specific document-heavy feature. This is normal and intentional, not a sign of confusion.

Transitions: This lesson closes the content. Students now have both the map and a compass for navigating it.

Key Principles:

- No database type is universally superior — each is optimal for its intended use case
- The boring default (PostgreSQL) is boring because it's reliably excellent for most problems
- Polyglot persistence is a mature engineering pattern, not a failure to commit to one technology

Examples:

- Scenario A: A social media app — PostgreSQL for users/posts, Redis for feed caching, a graph database if "suggested connections" becomes a core feature
- Scenario B: An e-commerce site — PostgreSQL for products/orders/users (relational, with clear relationships), Redis for shopping cart sessions and rate limiting
- Scenario C: An IoT platform collecting sensor readings from 10,000 devices — a columnar database for analytics, potentially Redis for real-time stream buffering

---

03.1 Knowledge Check 🧠

Learning Objectives:

- Demonstrate understanding of the four main database categories and their characteristics
- Identify the appropriate database type for a given scenario
- Distinguish between commonly confused concepts (e.g., SQLite vs. document databases, Redis as cache vs. primary store)

Question Format: Multiple choice

Questions:

1. Which of the following best describes a relational database?
    - a) Data is stored as self-contained JSON-like documents
    - b) Data is organized into tables with rows and columns, linked by keys ✓
    - c) Data is stored as key-value pairs in memory
    - d) Data is stored as nodes and edges
2. You are building a blog platform where each post can have a variable number of tags, an optional featured image, and nested comment threads. Which database type is the most natural fit?
    - a) Relational database (PostgreSQL)
    - b) Key-value store (Redis)
    - c) Document database (MongoDB) ✓
    - d) Columnar database (Cassandra)
3. What is Redis primarily optimized for?
    - a) Storing and querying complex relational data

    - b) Fast lookup and retrieval by a known key, often in memory ✓
    - c) Storing documents with flexible schemas
    - d) Analytical queries across large datasets

4. A student says "SQLite is a document database because it stores data in a single file." What is wrong with this statement?

    - a) Nothing — SQLite is a document database
    - b) SQLite is actually a key-value store
    - c) SQLite uses the relational model and SQL — it's file-based, but not a document database ✓
    - d) SQLite doesn't exist; the correct name is TinyDB

5. You need to implement a "people you may know" feature for a professional networking app, where the recommendation is based on shared connections up to 3 degrees away. Which database type is best suited for traversing these relationships efficiently?

    - a) Relational database
    - b) Document database
    - c) Key-value store
    - d) Graph database ✓

6. Which of the following is the most common PostgreSQL use case in modern web development?

    - a) Storing session data for fast access
    - b) Serving as the primary relational database for application data ✓
    - c) Streaming real-time events between microservices
    - d) Caching the results of expensive computations

7. What is a key tradeoff of relational databases compared to document databases?

    - a) Relational databases cannot store numeric data
    - b) Relational databases require a fixed schema upfront, while document databases allow flexible shapes ✓
    - c) Relational databases are always slower than document databases
    - d) Relational databases cannot handle relationships between records

8. Which scenario is NOT a good fit for Redis?

    - a) Caching the results of a slow database query
    - b) Storing user session tokens
    - c) Serving as the primary database for a user's complete order history ✓
    - d) Rate limiting API requests

9. A company runs a data warehouse that aggregates sales figures from millions of transactions to generate monthly reports. Queries typically read 2-3 columns across all rows. Which database type is optimized for this workload?

    - a) Document database (MongoDB)
    - b) Key-value store (Redis)

    - c) Relational database (PostgreSQL)
    - d) Columnar database (Cassandra) ✓

10. What does "polyglot persistence" mean?

    - a) A single database that supports multiple query languages
    - b) Using multiple database types together in one system, each for the problem it solves best ✓
    - c) A database that can store data in multiple human languages
    - d) Migrating from one database type to another

Evaluation Criteria:

- 9-10 correct: Excellent — solid grasp of the database landscape and tradeoffs
- 7-8 correct: Good — minor gaps in specific use cases or distinctions
- 5-6 correct: Needs review — revisit the sections on database types and the decision framework
- Below 5: Restart recommended — return to Section 01 and work through carefully

Score Interpretation: The goal is not to memorize which companies use which databases — it's to reason about data shapes and access patterns. If you missed scenario-based questions (2, 5, 8, 9), review the "Choosing the Right Database" lesson and focus on the decision framework.

Section 04: Outro

04.0 What's Next

Content Outline:

- Course recap: what students accomplished
    - Mapped the full database landscape: relational, document, key-value, graph, columnar
    - Understood TinyDB in context: it was a document database all along
    - Learned the SQL family: PostgreSQL, MySQL/MariaDB, SQL Server
    - Built a decision framework for choosing the right database type
- Connection to bigger picture
    - Databases are the foundation under almost every application that matters
    - Understanding the landscape makes every future database conversation clearer — whether reading documentation, choosing a stack, or debugging a production issue
    - The next course will go hands-on with relational databases and SQL: the concepts from this course (tables, rows, relationships, primary keys) will become concrete code
- Next steps and continued learning
    - Next: Gestionando tablas relacionadas con SQL (Skill 31) — JOINs, foreign keys, related tables
    - Later: ORM with SQLAlchemy in FastAPI (Skill 33) — connecting Python models to database tables
    - Explore: Official PostgreSQL documentation, Redis documentation, MongoDB University (free)
- Encouragement

- The database landscape can feel overwhelming at first — dozens of technologies, endless debates about which is best. The compass is always the same: what shape is your data, how will you access it, and what does your team already know? You now have that compass.

Note: This is conclusion only — no theory, exercises, or assessment.
